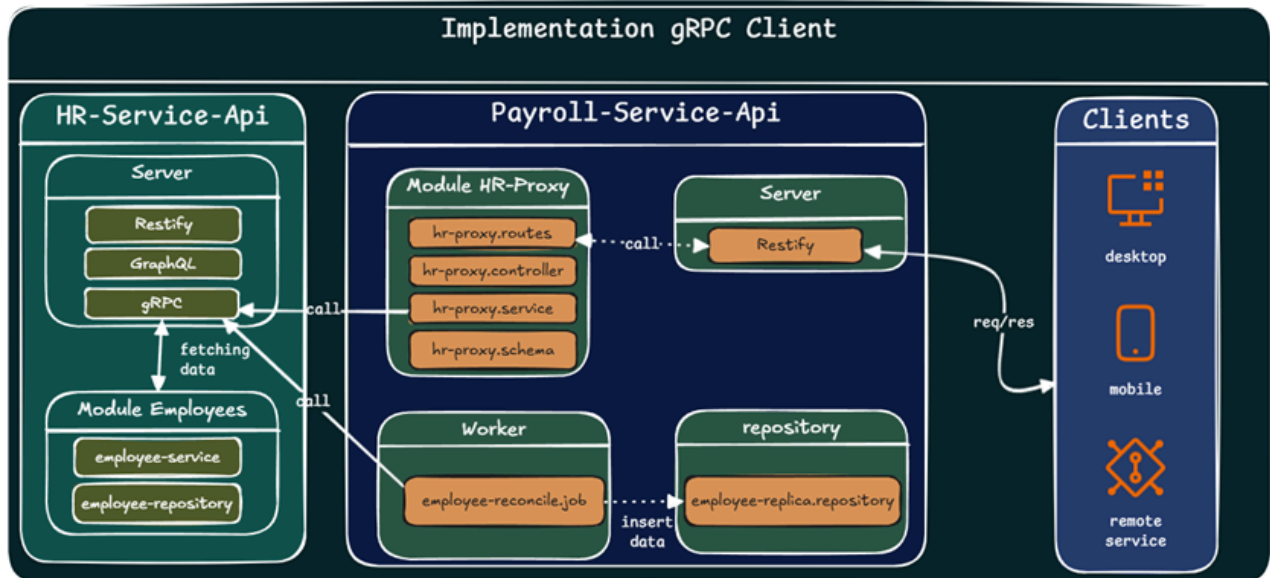


01-03. gRPC Client

Berikut kita akan implementasikan bagaimana call gRPC server di project **payroll-service-api**.



Kita akan gunakan dua metode call hr-service-api (gRPC server) oleh payroll-service-api (grpc-client) :

1. Kita akan call method gRPC di server, client-stub dibaca controller lalu route endpoint (fastify) dicall dan ditampilkan di client (web/mobile).
2. Kita gunakan worker terpisah untuk memindahkan data employee secara streaming dari gRPC Server ke table replica employee di **payroll_db**.

Step By Step

A. Call gRPC With Restify

1. Create employee.proto generator di payroll-service-api.

Gunakan langkah yang sama seperti di materi sebelumnya untuk generate employee-protoc.

- Copykan file **hr-service-api/protos/employee.proto** di project hr-service-api, dan paste di folder **payroll-service-api/protos/employee.proto**.
- Copykan file **hr-service-api/scripts/generate-proto.js** ke **payroll-service-api/scripts/generate-proto.js**.
- Run generator script dan pastikan employee.proto sukses digenerate.

2. Create file hr-service.client

Berikutnya create file **hr-service-client.ts**. File ini akan call client-stub gRPC di hr-service0api.

```
//src/grpchr-proxy.routes.ts
import { credentials, status as grpcStatus, type ServiceError } from
  '@grpc/grpc-js';
import {
  EmployeeServiceClient,
  type DepartmentResponse,
  type EmployeeResponse
} from './generated/employee';

const HR_SERVICE_GRPC_URL = process.env.HR_SERVICE_GRPC_URL ??
  'localhost:50051';

let client: EmployeeServiceClient | undefined;

function getClient(): EmployeeServiceClient {
  if (!client) {
    client = new EmployeeServiceClient(HR_SERVICE_GRPC_URL,
      credentials.createInsecure());
  }
  return client;
}

export function closeHrServiceClient(): void {
  client?.close();
  client = undefined;
}

export function getEmployee(employeeId: number): Promise<EmployeeResponse |
  null> {
  return new Promise((resolve, reject) => {
    getClient().getEmployee({ employeeId: employeeId }, (err:
      ServiceError | null, res) => {
      if (err) {
        if (err.code === grpcStatus.NOT_FOUND) return resolve(null);
        return reject(err);
      }
      resolve(res ?? null);
    });
  });
}

export function listEmployeesForPayroll(
  departmentId?: number,
  employmentStatus?: string,
): Promise<EmployeeResponse[]> {
  return new Promise((resolve, reject) => {
```

```
const results: EmployeeResponse[] = [];
const call = getClient().getEmployeesForPayroll({
  departmentId: departmentId ?? 0,
  employmentStatus: employmentStatus ?? '',
});
//alasan kenapa kita gunakan emitter adalah karena data yg dikirim
dari grpc hr-service-api
//dikirim streaming, maka kita simpan dulu di emitter
call.on('data'), di push dulu ke array
// baru setelah selesai di return kan via promise resolve
call.on('data', (res: EmployeeResponse) => results.push(res));
call.on('end', () => resolve(results));
call.on('error', (err) => reject(err));
});
}

export function streamEmployeesForPayroll(
  departmentId: number | undefined,
  onEmployee: (employee: EmployeeResponse) => Promise<void> | void,
): Promise<void> {
  return new Promise((resolve, reject) => {
    const call = getClient().getEmployeesForPayroll({
      departmentId: departmentId ?? 0,
      employmentStatus: '',
    });

    call.on('data', async (res: EmployeeResponse) => {
      call.pause();
      try {
        await onEmployee(res);
        call.resume();
      } catch (err) {
        call.destroy(err as Error);
      }
    });
    call.on('end', () => resolve());
    call.on('error', (err) => reject(err));
  });
}

export function listDepartments(): Promise<DepartmentResponse[]> {
  return new Promise((resolve, reject) => {
    const results: DepartmentResponse[] = [];
    const call = getClient().listDepartments({});

    call.on('data', (res: DepartmentResponse) => results.push(res));
    call.on('end', () => resolve(results));
    call.on('error', (err) => reject(err));
  });
}
```

```
export function createDepartment(departmentName: string):
Promise<DepartmentResponse> {
  return new Promise((resolve, reject) => {
    getClient().createDepartment({ departmentName: departmentName },
(err: ServiceError | null, res) => {
      if (err) return reject(err);
      resolve(res!);
    });
  });
}

export function updateDepartment(departmentId: number, departmentName:
string): Promise<DepartmentResponse> {
  return new Promise((resolve, reject) => {
    getClient().updateDepartment(
      { departmentId: departmentId, departmentName: departmentName },
      (err: ServiceError | null, res) => {
        if (err) return reject(err);
        resolve(res!);
      },
    );
  });
}

export function deleteDepartment(departmentId: number): Promise<boolean> {
  return new Promise((resolve, reject) => {
    getClient().deleteDepartment({ departmentId: departmentId }, (err:
ServiceError | null, res) => {
      if (err) return reject(err);
      resolve(Boolean(res?.success));
    });
  });
}
```

3. Create module hr-proxy.

- Create hr-proxy-service.

```
//src/modules/hr-proxy.service.ts
import { ApiError } from '../../common/utils/api-error';
import { DepartmentResponse, EmployeeResponse } from
'../../grpc/generated/employee';
import { getEmployee, listDepartments, createDepartment } from
'../../grpc/hr-service.client';
//import type { DepartmentMessage, EmployeeMessage } from
'../../grpc/hr.types';

export async function getEmployeeById(id: number):
```

```
Promise<EmployeeResponse> {
  const employee = await getEmployee(id);
  if (!employee) {
    throw ApiError.notFound(`Employee dengan id ${id} tidak ditemukan di hr-service`);
  }
  return employee;
}

export async function getAllDepartments(): Promise<DepartmentResponse[]>
{
  return listDepartments();
}

export async function createNewDepartment(departmentName: string):
Promise<DepartmentResponse> {
  return createDepartment(departmentName);
}
```

- Create hr-proxy-schema

```
//src/modules/hr-proxy.schema.ts
import { Type, type Static } from '@sinclair/typebox'

export const employeeIdParamSchema = Type.Object({
  id: Type.Integer({ minimum: 1 }),
})
export type EmployeeIdParam = Static<typeof employeeIdParamSchema>

// baru nambahin
export const createDepartmentBodySchema = Type.Object({
  department_name: Type.String({ minLength: 1, maxLength: 30 }),
}, { additionalProperties: false })
export type CreateDepartmentInput = Static<typeof
createDepartmentBodySchema>
```

- Create hr-proxy.controller

```
//src/modules/hr-proxy.controller.ts
import type { FastifyReply, FastifyRequest } from 'fastify';
import { sendSuccess } from '../../common/utils/api-response';
import * as hrProxyService from './hr-proxy.service';
import type { CreateDepartmentInput, EmployeeIdParam } from './hr-
proxy.schema';

export async function getEmployee(
```

```
    request: FastifyRequest<{ Params: EmployeeIdParam }>,
    reply: FastifyReply,
  ): Promise<void> {
    const employee = await
hrProxyService.getEmployeeById(request.params.id);
    sendSuccess(reply, employee, 'Employee berhasil diambil dari hr-
service-api (gRPC)');
  }

export async function listDepartments(
  _request: FastifyRequest,
  reply: FastifyReply,
): Promise<void> {
  const departments = await hrProxyService.getAllDepartments();
  sendSuccess(reply, departments, 'Daftar department berhasil diambil
dari hr-service-api (gRPC)');
}

export async function createDepartments(
  request: FastifyRequest<{ Body: CreateDepartmentInput }>,
  reply: FastifyReply,
): Promise<void> {
  const department = await
hrProxyService.createNewDepartment(request.body.department_name);
  sendSuccess(reply, department, 'Create department sukses ke hr-
service-api (gRPC)', 201);
}
```

- Create hr-proxy.routes

```
//src/modules/hr-proxy.routes.ts
import type { FastifyPluginAsync } from 'fastify';
import * as hrProxyController from './hr-proxy.controller';
import { createDepartmentBodySchema, employeeIdParamSchema } from './hr-
proxy.schema';

export const hrProxyRoutes: FastifyPluginAsync = async (fastify) => {
  fastify.get(
    '/employees/:id',
    { schema: { params: employeeIdParamSchema } },
    hrProxyController.getEmployee,
  );

  fastify.get('/departments', hrProxyController.listDepartments);

  fastify.post(
    '/departments',
    { schema: { body: createDepartmentBodySchema } },
    hrProxyController.createDepartments,
  );
}
```

```
);  
};
```

4. Create index routes

```
// src/routes/index.ts  
import type { FastifyPluginAsync } from 'fastify';  
import { payrollRunRoutes } from '../modules/payroll-run/payroll-run.routes';  
import { sagaRoutes } from '../modules/saga/saga.routes';  
import { hrProxyRoutes } from '../modules/hr-proxy/hr-proxy.routes';  
  
export const apiRoutes: FastifyPluginAsync = async (fastify) => {  
  await fastify.register(payrollRunRoutes, { prefix: '/payroll-runs' });  
  await fastify.register(sagaRoutes, { prefix: '/saga-instances' });  
  await fastify.register(hrProxyRoutes, { prefix: '/hr' });  
};
```

5. Register ApiRoutes di app.ts.

Register route di **app.ts**

```
// src/app.ts  
import cors from '@fastify/cors';  
import helmet from '@fastify/helmet';  
import type { JsonSchemaToTsProvider } from '@fastify/type-provider-json-schema-to-ts';  
import Fastify from 'fastify';  
import { errorHandler, notFoundHandler } from '../common/handlers/error-handler';  
import { env } from '../config/env';  
import { apiRoutes } from '../routes';  
  
export async function createApp() {  
  const app = Fastify({  
    logger: env.NODE_ENV !== 'test',  
  }).withTypeProvider<JsonSchemaToTsProvider>();  
  
  await app.register(helmet);  
  await app.register(cors);  
  
  app.setNotFoundHandler(notFoundHandler);  
  app.setErrorHandler(errorHandler);  
  
  app.get('/health', async () => ({ success: true, message: 'payroll-service-'
```

```
api is healthy' }));

await app.register(apiRoutes, { prefix: env.API_PREFIX });

return app;
}
```

6. Tambahkan close hr-service-client di server.ts

```
//src/server.ts
import { createApp } from './app';
import { env } from './config/env';
import { checkDatabaseConnection, pool } from './db';
import { closeHrServiceClient } from './grpc/hr-service.client';

async function bootstrap(): Promise<void> {
  await checkDatabaseConnection();
  console.log('Koneksi PostgreSQL OK');

  const app = await createApp();
  await app.listen({ port: env.PORT, host: '0.0.0.0' });
  console.log(`REST server berjalan di http://localhost:${env.PORT}`);
  console.log(`Health check : http://localhost:${env.PORT}/health`);
  console.log(`API prefix: ${env.API_PREFIX}`);

  const shutdown = async (signal: string) => {
    console.log(`\n${signal} diterima, mematikan server dengan aman...`);

    await app.close();
    console.log('REST server ditutup.');
```

```
    //close koneksi channel grpc
    closeHrServiceClient();
    console.log('Channel gRPC ke hr-service-api ditutup.');
```

```
    await pool.end();
    console.log('Pool database ditutup. Sampai jumpa!');
    process.exit(0);
  };

  process.on('SIGINT', () => void shutdown('SIGINT'));
  process.on('SIGTERM', () => void shutdown('SIGTERM'));
}

bootstrap().catch((err) => {
  console.error('Gagal start aplikasi:', err);
  process.exit(1);
});
```

```
});
```

7. Run fastify server.

```
npm run dev
```